

2.构建DOM与CSSOM

一、DOM树的构建过程 (Incremental)

当浏览器从服务器接收到HTML文档的字节数据后，它会立即开始处理，这个过程是**渐进式**的，意味着浏览器无需等待整个文档加载完毕就可以开始解析和渲染页面。

构建流程如下：

1. **字节 (Bytes) → 字符 (Characters)**: 浏览器根据文件指定的编码（例如 `UTF-8`）将原始的字节数据转换为字符。
2. **字符 (Characters) → 令牌 (Tokens)**: 浏览器将字符串形式的字符转换为W3C HTML5标准所规定的各种令牌 (Token)，例如 `<html>`、`<body>` 等。每个令牌都具有特殊的含义和一组属性。这个过程被称为“词法分析”或“令牌化”。
3. **令牌 (Tokens) → 节点 (Nodes)**: 经过词法分析后，令牌会被转换成定义了其属性和规则的“对象”（即节点）。
4. **节点 (Nodes) → DOM树 (DOM Tree)**: 由于HTML中的元素存在嵌套关系，这些节点之间会根据这种关系链接成一个树形数据结构，这就是文档对象模型 (DOM)。

关键特性：渐进式构建

DOM的构建过程是自上而下、循序渐进的。浏览器每接收到一部分HTML，就会解析并生成对应的DOM节点，并将其添加到DOM树中。这使得浏览器可以在接收到全部HTML之前，就开始渲染页面的已就绪部分，这也是为什么我们有时会看到网页内容从上到下一点点加载显示出来的原因。

二、CSSOM树的构建过程 (Blocking)

与构建DOM类似，当浏览器遇到CSS代码（无论是外部CSS文件、`style` 标签还是内联样式）时，也会进行类似的处理。

构建流程如下：

1. **字节 (Bytes) → 字符 (Characters)**: 将CSS文件字节转换为字符。
2. **字符 (Characters) → 令牌 (Tokens)**: 将字符转换为CSS令牌。
3. **令牌 (Tokens) → 节点 (Nodes)**: 将令牌转换为包含样式信息的CSS节点。
4. **节点 (Nodes) → CSSOM树 (CSSOM Tree)**: 将CSS节点聚合成一个树形结构，即CSS对象模型 (CSSOM)。

关键特性：渲染阻塞

与DOM不同，CSSOM的构建是**渲染阻塞 (Render-Blocking)**的。这意味着在CSSOM树完全构建完成之前，浏览器不会进行后续的渲染树构建、布局和绘制工作。

三、为何DOM是渐进式的，而CSSOM是阻塞性的？

理解这个差异的核心在于**样式的继承和覆盖规则**。

- **对于DOM：**一个父节点的结构并不会被其后的兄弟节点或子节点所改变。解析到 `<div>` 时，浏览器就可以确定这是一个 `div` 元素，无需关心它后面会出现什么内容。因此，它可以一块一块地构建和显示。
- **对于CSSOM：**CSS的规则是层叠的。一个后面定义的样式规则可能会覆盖或修改前面定义的规则。

举个例子：

代码块

```
1  <head>
2    <link rel="stylesheet" href="style.css">
3  </head>
4  <body>
5    <div class="box">这是一个盒子</div>
6  </body>
```

代码块

```
1  /* style.css */
2  body {
3    font-size: 16px;
4  }
5
6  .box {
7    color: blue;
8  }
9
10 /* ... 文件后面可能还有很多其他规则 ... */
11
12 div {
13   color: red; /* 这个规则会覆盖 .box 的颜色 */
14 }
```

在这个例子中，`.box` 元素的颜色最初被设为蓝色，但随后又被 `div` 选择器覆盖为红色。

如果浏览器不等 `style.css` 文件完全下载和解析完毕就开始渲染页面，它可能会先用蓝色渲染 `.box`，等解析到文件末尾时发现颜色应该是红色，于是不得不重新绘制这个元素。这种行为会导致页面内容的**“样式闪烁” (Flash of Unstyled Content, FOUC)**，用户体验极差。

为了避免这种情况，浏览器选择了一种更稳妥的策略：**等待所有CSS都加载和解析完毕，构建出完整的、最终的CSSOM树之后，再用这个最终的样式信息去渲染整个页面。** 这就是CSS之所以会“阻塞渲染”的原因。

小结与优化启示

- **DOM是增量的：**HTML的解析和DOM构建可以流式进行，无需等待整个文档。
- **CSSOM是阻塞的：**浏览器必须拥有完整的CSSOM才能进入下一渲染阶段，以确保元素样式的正确性。

这个基础知识直接引出了前端优化的一个核心原则：**尽快、尽早地加载CSS，并减少CSS文件的大小**，以缩短CSSOM的构建时间，从而缩短渲染被阻塞的时间，让用户能更快地看到页面的首次绘制。